

Behavioural-Based Intrusion Detection System Using Machine Learning

B. Samiksha, Shraddha V T, Shreshtha S S, Siddhi Shetty

Department of Computer Science and Engineering (IoT and Cybersecurity with Blockchain Technology)

Mangalore Institute of Technology and Engineering, Mangalore, India

Under the guidance of Dr. Shivananda V. Seeri, Head of the Department

Abstract—The scale and variety of cyberattacks targeting modern networks continue to increase, and conventional intrusion detection methods are struggling to keep pace with threats that are new or constantly changing. This work proposes a Behavioural-Based Intrusion Detection System built on a combination of Isolation Forest and XGBoost to detect and classify network-level attacks. Network traffic is examined for abnormal behaviour, with detected anomalies sorted into the attack categories defined by the CICIDS2018 dataset. The system includes a real-time dashboard for ongoing traffic monitoring and alerting, as well as a report history module that supports later review and auditing of detected incidents. An Isolation Forest baseline was first evaluated on its own and then extended into a two-stage Isolation Forest plus XGBoost pipeline; the combined model raised held-out test accuracy from 78.58% to 99.98% and F1-score from 0.61 to above 0.99. Taken together, the behavioural analysis engine, ML-driven classification, live monitoring, and historical logging form a practical framework for tracking network threats as they evolve.

Keywords—Intrusion Detection System, Network Intrusion Detection, Anomaly Detection, Behavioural-Based Detection, Machine Learning, Isolation Forest, XGBoost, Network Security, Cybersecurity, CICIDS2018, Attack Classification, Real-Time Monitoring

I. INTRODUCTION

The growing scale and diversity of cyberattacks is outpacing what conventional intrusion detection systems can reliably catch. Signature-based tools work only against attack patterns they already know, which leaves them exposed to anything novel or slightly modified. As attackers keep finding ways around rule-based defences, analysing network traffic directly has become one of the more dependable ways to spot malicious behaviour. A persistent problem in the field is the lag between when an attack happens and when it's actually detected — closing that gap is central to any move toward proactive security.

Existing IDS approaches fall short in several respects. Unknown or previously unseen attacks routinely slip past them. False alarms are common enough to undermine confidence in their output. Many systems process traffic in batches rather than continuously, which delays detection. Historical logging is often too thin to support proper post-incident investigation. And because traffic patterns vary so much across different network environments, consistent detection across systems is hard to achieve.

To address these gaps, this paper introduces a Behavioural-Based Intrusion Detection System that pairs Isolation Forest with XGBoost. Isolation Forest handles unsupervised anomaly detection on traffic flows, flagging behaviour that departs from normal patterns, while XGBoost performs supervised classification to sort flagged anomalies into specific attack types aligned with the CICIDS2018 taxonomy. The system's contributions include: a two-stage design that pairs unsupervised anomaly detection with supervised classification; a real-time dashboard for live traffic analysis and threat visualization; a historical reporting module for forensic review and pattern analysis after the fact; automated extraction of features such as inter-arrival timing and packet-level statistics; and a tuned model configuration that achieves strong detection accuracy while keeping false positives to a minimum. Combined, these elements — behavioural analysis, ML-based classification, real-time monitoring, and audit-ready reporting — offer a practical way to detect and track network threats across varied environments.

II. RELATED WORK

Sharafaldin et al. [1] built the CICIDS2018 dataset to address the shortcomings of older intrusion-detection benchmarks such as KDD'99 and NSL-KDD, which had grown outdated and lacked realistic traffic diversity. Their dataset simulates a mid-sized organisational network attacked from a separate infrastructure, producing labelled flow records for benign traffic and seven attack families computed with CICFlowMeter. It is now one of the most widely used benchmarks for evaluating network-based anomaly detection, and it forms the training and evaluation basis for the system described in this paper, but the dataset paper itself does not propose a detection pipeline — it only supplies the data that later systems, including this one, must learn from.

Liu et al. [2] introduced the Isolation Forest algorithm, which detects anomalies by measuring how few random partitions are needed to isolate a data point rather than by modelling what normal data looks like. Because it needs no labelled attack examples and scales close to linearly with dataset size, it has become a popular first-stage detector for network traffic. Its main limitation for intrusion detection is that it only produces a numeric anomaly score — it cannot say what kind of attack a flagged flow belongs to, which is exactly the gap a downstream classifier is needed to close.

Miran and Kadir [3] compared two supervised models (Random Forest, XGBoost) against two unsupervised models (Isolation Forest, One-Class SVM) on the CICIDS2017 dataset and found that the supervised classifier reached very high accuracy on attack types present in training, while the

unsupervised models generalised better to behaviour that was not explicitly labelled. Their study evaluated the two families of algorithms side by side rather than combining them into a single pipeline, so a system that still needs labelled attack data for known threats and unsupervised screening for unknown ones has to be assembled separately.

Marteau et al. [4] proposed a Hybrid Isolation Forest that corrects a known bias in the original algorithm and adds a limited amount of supervised guidance, testing it on the ISCX intrusion-detection benchmark. Their results showed a measurable improvement over the standard Isolation Forest and competitive performance against SVM baselines. The work stays focused on improving the anomaly-scoring stage itself, though, and does not extend to fine-grained attack-category classification or a deployable monitoring system.

Itasoy et al. [5] combined Isolation Forest and XGBoost in a different security domain, pairing the two to detect ransomware from Windows file-system activity rather than network flows. Isolation Forest was used to flag unusual file operations, and XGBoost then classified the flagged activity, reducing false alarms relative to using either model alone. Their results support the idea that pairing an unsupervised anomaly detector with a supervised classifier generalises well across security problems, though their pipeline targets host-level file activity and does not include a live network dashboard or a persistent incident-history module.

Boukhalfa et al. [6] used a Long Short-Term Memory network to model the sequential structure of network traffic, allowing the system to recognise attack patterns that unfold over several packets rather than in a single flow snapshot. Sequence modelling of this kind can capture behaviour that flow-level statistics miss, but it comes with a heavier training and inference cost, which complicates deployment on the kind of lightweight, continuously running monitor this project targets.

Thirimanne et al. [7] built a real-time detection system around a deep neural network trained on the NSL-KDD dataset, exposing it through a REST API and pairing it with a live feature extractor that sniffs traffic between the gateway router and the local network. The architecture is close in spirit to the pipeline used here, but the detector is purely supervised and trained on an older dataset, so it cannot flag attack behaviour that was not represented during training, and the system does not include graduated response modes or automatic mitigation.

Taken together, these works cover pieces of the problem — a modern benchmark dataset, an unsupervised anomaly detector, a supervised classifier, sequence-aware deep learning, and real-time deployment — but no single system among them combines unsupervised behavioural screening with supervised, category-level classification on CICIDS2018, wraps the result in a live dashboard, and keeps a queryable history of past incidents for later review. Closing that combined gap is the specific aim of the system presented in this paper.

III. SYSTEM ARCHITECTURE

The proposed system is organised into four cooperating layers: a sensing layer that captures live network activity, a machine-learning layer that scores and classifies that activity, a persistence layer that stores results for later review, and a presentation layer that surfaces everything to an analyst in real time. Fig. 1 shows how traffic moves through these layers, from the moment a packet is captured to the point where it either passes through quietly or triggers an alert and, if Protection mode is active, an automatic response.

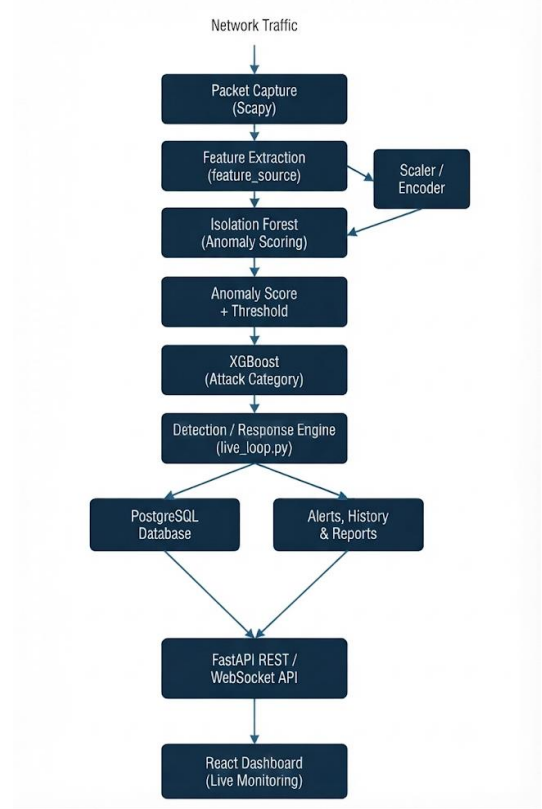


Fig. 1. End-to-end architecture of the behavioural IDS pipeline.

A. Data Collection and Preprocessing

The system is trained offline on the CICIDS2018 flow-level dataset, which is first cleaned and prepared before any model sees it: numeric fields are scaled, categorical fields are encoded, records are shuffled to remove ordering bias, and the data is split into training, validation, and test sets in a 60/20/20 ratio. At inference time, the same feature set has to be reproduced from live traffic, so a feature-extraction module (`feature_source`) derives the same thirty flow-level attributes — duration, packet and byte counts, inter-arrival timing, and related statistics — from packets captured with Scapy and connection information collected through psutil. The trained model, the fitted scaler, and the label encoder are all saved as .pkl artifacts so that live traffic is transformed in exactly the same way the training data was, which keeps the live pipeline consistent with what the models actually learned.

B. Behavioural Anomaly Detection Engine

Isolation Forest is trained only on the characteristics of the traffic itself, without needing attack labels, by repeatedly

partitioning the feature space at random and measuring how many partitions it takes to isolate each point. Points that separate out quickly — because they sit far from the bulk of ordinary traffic — receive a higher anomaly score. A configurable threshold on this score marks a flow as suspicious. Because the model never relies on a catalogue of known attack signatures, it is able to flag traffic that departs from normal behaviour even when that traffic doesn't match any attack pattern the system has seen labelled examples of before.

C. Attack Category Classification

A flow that clears the anomaly threshold is not immediately reported as a generic "anomaly" — it is passed on to an XGBoost classifier [8] trained on the labelled attack categories in CICIDS2018, which assigns it to a specific category such as Denial-of-Service, Distributed Denial-of-Service, brute-force, botnet activity, or another catalogued class. Running the heavier, labelled classifier only on flows that Isolation Forest has already flagged keeps the overall pipeline efficient, since the majority of ordinary traffic never reaches this stage, and it gives an analyst a concrete category to act on rather than a bare anomaly flag. This two-stage design is what lets the system combine the broad, label-free coverage of unsupervised detection with the precision of supervised classification, without paying the full computational cost of running the classifier on every single flow.

D. Real-Time Detection and Response Engine

A background detection loop (`live_loop.py`) re-scores incoming flows roughly every one to two seconds, keeping the dashboard close to real time. The system can run in one of three modes: OFF, where no scoring takes place; Detection, where anomalous flows are scored, classified, and surfaced as alerts; and Protection, where the same detection pipeline additionally triggers an automatic response. In Protection mode, and where the running process has sufficient privileges, the system can block an offending IP address or flag a suspicious process using platform-appropriate tools — `netsh` on Windows or `iptables` on Linux. This closes the gap identified in Section I, where most research prototypes stop at raising an alert and leave the actual mitigation step to a human.

E. Storage, Alerting and Report History

Every flagged event — its anomaly score, assigned attack category, the features that contributed most to the decision, and any automatic response that was taken — is written to a PostgreSQL database. A report-history module built on top of this storage lets an analyst filter and revisit past incidents instead of only seeing whatever is currently on screen, which is what makes the system usable for after-the-fact audits and pattern analysis rather than just live monitoring.

F. Presentation Layer

A FastAPI backend exposes REST endpoints for historical queries and a WebSocket channel that streams live anomaly scores and alerts as they are generated. The frontend, a React and TypeScript dashboard referred to internally as SENTRY,

renders the live anomaly-score graph, the current operating mode, overall system status, and a running alert feed, and it is complemented by desktop and email notifications so that an analyst does not have to keep the dashboard open at all times to be informed of a new incident.

IV. IMPLEMENTATION

The system is implemented as a full-stack application rather than a standalone research script, which was a deliberate choice to keep the detection pipeline usable outside of a notebook environment. Table I summarises the technology choices for each layer.

TABLE I TECHNOLOGY STACK

Layer	Technology
Frontend Dashboard	React, TypeScript
Backend / Inference Service	FastAPI (Python)
Anomaly Detection Model	Isolation Forest (scikit-learn)
Attack Classification Model	XGBoost
Live Traffic Capture	Scapy, psutil
Real-Time Channel	WebSocket
Persistent Storage	PostgreSQL
Alerting	Desktop and email notifications
Dataset	CICIDS2018

Algorithm 1 sketches the live scoring and response loop that ties the machine-learning layer to the storage and alerting layers. The loop is intentionally simple so that it can run continuously with a small, predictable resource footprint.

Algorithm 1 Live Anomaly Scoring and Response

```

Input: live flow F, threshold  $\theta$ , mode M
Output: alert record or none
features  $\leftarrow$  feature_source(F)
scaled  $\leftarrow$  Scaler.transform(features)
score  $\leftarrow$  IsolationForest.score(scaled)
if score >  $\theta$  then
    category  $\leftarrow$  XGBoost.predict(scaled)
    topFeatures  $\leftarrow$  ExplainTop(scaled, category)
    StoreAlert(F, score, category, topFeatures)
    if M == "Protection" then
        BlockOrFlag(F.sourceIP)
    end if
    PushToWebSocket(alert)
else
    continue monitoring
end if

```

The FastAPI service loads the persisted model, scaler, and label encoder once at startup rather than on every request, which keeps inference latency low and avoids repeated disk access. The scoring loop itself runs asynchronously so that the dashboard's WebSocket connection and any REST queries against the history database remain responsive even while traffic is actively being analysed. Failed or dropped connections are retried rather than allowed to silently halt monitoring, since a detection system that stops watching

traffic without any indication of doing so would defeat its own purpose.